LAB 6:
LCD Controller Design Using FSM in VHDL and Implementation on FPGA

Objectives

- To understand the operation of an LCD controller through Finite State Machine (FSM) design.
- To implement the FSM-based LCD controller on an FPGA board and display characters on the LCD.
- To analyze the role of timing, control signals, and state transitions in LCD interfacing.

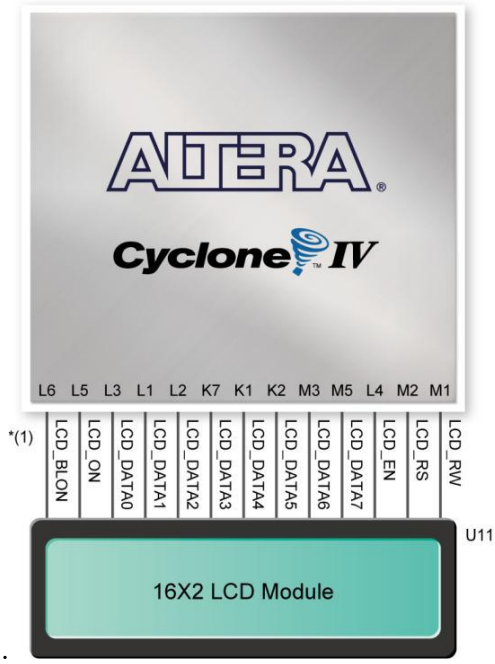
Pre-Lab

Part 1 – Introduction to LCD

LCD stands for Liquid Crystal Display, commonly used to display alphanumeric data. Characters are represented using their ASCII codes (8-bit). The LCD module used in FPGA boards, such as DE2-115, is typically based on the HD44780 controller, which includes built-in character fonts and interprets command codes for initialization and display control.

The controller requires a defined sequence of initialization commands and timing delays, which are managed here using a Finite State Machine (FSM) written in VHDL. The FSM controls signal like ***RS, RW, E***, and ***lcd_data***, ensuring that LCD setup and data display occur in the correct order.

A schematic diagram of the LCD module showing connections to the Cyclone IV E FPGA is provided in Figure 6-1



(Note: The LCD modules on DE2-115 boards do not have a backlight. The LCD_BLON signal is not used.)

Pin Assignments for LCD Module

Signal Name	*FPGA Pin No.*	*Description*
LCD_DATA[7]	PIN_M5	LCD Data[7]
LCD_DATA[6]	PIN_M3	LCD Data[6]
LCD_DATA[5]	PIN_K2	LCD Data[5]
LCD_DATA[4]	PIN_K1	LCD Data[4]
LCD_DATA[3]	PIN_K7	LCD Data[3]
LCD_DATA[2]	PIN_L2	LCD Data[2]
LCD_DATA[1]	PIN_L1	LCD Data[1]
LCD_DATA[0]	PIN_L3	LCD Data[0]
LCD_EN	PIN_L4	LCD Enable
LCD_RW	PIN_M1	LCD Read/Write Select, 0 = Write, 1 = Read
LCD_RS	PIN_M2	LCD Command/Data Select, 0 = Command, 1 = Data
LCD_ON	PIN_L5	LCD Power ON/OFF
LCD_BLON	PIN_L6	LCD Back Light ON/OFF

Pin Assignments for LCD Module

Pin no	Name	Function	Description
1	Vss	Power supply	Ground
2	Vdd	Power supply	+5v
3	Vo	Contrast adjust	0-5v
4	RS	Command	Register select
5	RW	Command	Read /Write
6	E	Command	Enable
7	D0	Input/Output	Data(LSB)
8	D1	Input/Output	Data
9	D2	Input/Output	Data
10	D3	Input/Output	Data
11	D4	Input/Output	Data
12	D5	Input/Output	Data
13	D6	Input/Output	Data
14	D7	Input/Output	Data(MSB)
15	A	LED BKL	+5v
16	K	LED BKL	Ground

Pre- Lab Task:

You are given two VHDL code files that implement the ***LCD_Controller*** and its wrapper file. Create a new VHDL project and test the code on FPGA.

Code for LCD_Controller

```
LIBRARY ieee;
USE ieee.std_logic_1164.ALL;


ENTITY lcd_controller IS
  PORT(
    clk        : IN    STD_LOGIC;  --system clock
    reset_n    : IN    STD_LOGIC;  --active low reinitializes lcd
    rw, rs, e  : OUT   STD_LOGIC;  --read/write, setup/data, and enable for lcd
    lcd_data   : OUT   STD_LOGIC_VECTOR(7 DOWNTO 0); --data signals for lcd
    line1_buffer : IN STD_LOGIC_VECTOR(127 downto 0); -- Data for the top line of the LCD
    line2_buffer : IN STD_LOGIC_VECTOR(127 downto 0)); -- Data for the bottom line of the LCD
END lcd_controller;


ARCHITECTURE controller OF lcd_controller IS
  TYPE CONTROL IS(power_up, initialize, RESETLINE, line1, line2, send);
  SIGNAL    state      : CONTROL;
  CONSTANT  freq       : INTEGER := 100; --system clock frequency in MHz
  SIGNAL    ptr        : natural range 0 to 16 := 15; -- To keep track of what character we are up to
  SIGNAL    line       : STD_LOGIC := '1';
BEGIN


 PROCESS(clk)
   VARIABLE clk_count : INTEGER := 0; --event counter for timing
 BEGIN
 IF(clk'EVENT and clk = '1') THEN

     CASE state IS

       --wait 50 ms to ensure Vdd has risen and required LCD wait is met
       WHEN power_up =>
         IF(clk_count < (50000 * freq)) THEN    --wait 50 ms
           clk_count := clk_count + 1;
           state <= power_up;
         ELSE                                   --power-up complete
           clk_count := 0;
           rs <= '0';
           rw <= '0';
           lcd_data <= "00110000";
           state <= initialize;
         END IF;

       --cycle through initialization sequence
       WHEN initialize =>
         clk_count := clk_count + 1;
         IF(clk_count < (10 * freq)) THEN       --function set
           lcd_data <= "00111100";      --2-line mode, display on
           --lcd_data <= "00110100";    --1-line mode, display on
           --lcd_data <= "00110000";    --1-line mdoe, display off
           --lcd_data <= "00111000";    --2-line mode, display off
           e <= '1';
           state <= initialize;
         ELSIF(clk_count < (60 * freq)) THEN    --wait 50 us
           lcd_data <= "00000000";
           e <= '0';
           state <= initialize;
         ELSIF(clk_count < (70 * freq)) THEN    --display on/off control
           lcd_data <= "00001100";      --display on, cursor off, blink off
           --lcd_data <= "00001101";    --display on, cursor off, blink on
           --lcd_data <= "00001110";    --display on, cursor on, blink off
           --lcd_data <= "00001111";    --display on, cursor on, blink on
           --lcd_data <= "00001000";    --display off, cursor off, blink off
           --lcd_data <= "00001001";    --display off, cursor off, blink on
           --lcd_data <= "00001010";    --display off, cursor on, blink off
           --lcd_data <= "00001011";    --display off, cursor on, blink on
           e <= '1';
           state <= initialize;
```

```
ELSIF(clk_count < (130 * freq)) THEN   --display clear
  lcd_data <= "00000001";
  e <= '1';
  state <= initialize;
ELSIF(clk_count < (2130 * freq)) THEN  --wait 2 ms
  lcd_data <= "00000000";
  e <= '0';
  state <= initialize;
ELSIF(clk_count < (2140 * freq)) THEN  --entry mode set
  lcd_data <= "00000110";      --increment mode, entire shift off
  --lcd_data <= "00000111";    --increment mode, entire shift on
  --lcd_data <= "00000100";    --decrement mode, entire shift off
  --lcd_data <= "00000101";    --decrement mode, entire shift on
  e <= '1';
  state <= initialize;
ELSIF(clk_count < (2200 * freq)) THEN  --wait 60 us
  lcd_data <= "00000000";
  e <= '0';
  state <= initialize;
ELSE                                   --initialization complete
  clk_count := 0;
  state <= RESETLINE;
END IF;

WHEN line2 =>
    line <= '0';
    lcd_data <= line2_buffer(ptr*8 + 7 downto ptr*8);
    rs <= '1';
    rw <= '0';
    clk_count := 0;
    state <= send;

   WHEN resetline =>
     ptr <= 16;
     if line = '1' then
       lcd_data <= "10000000";
       rs <= '0';
       rw <= '0';
       clk_count := 0;
       state <= send;
     else
       lcd_data <= "11000000";
       rs <= '0';
       rw <= '0';
       clk_count := 0;
       state <= send;
     end if;

   WHEN line1 =>
       line <= '1';
       lcd_data <= line1_buffer(ptr*8 + 7 downto ptr*8);
       rs <= '1';
       rw <= '0';
       clk_count := 0;
       line <= '1';
       state <= send;

WHEN send =>
   IF(clk_count < (50 * freq)) THEN  --do not exit for 50us
      IF(clk_count < freq) THEN      --negative enable
       e <= '0';
      ELSIF(clk_count < (14 * freq)) THEN  --positive enable half-cycle
       e <= '1';
      ELSIF(clk_count < (27 * freq)) THEN  --negative enable half-cycle
       e <= '0';
      END IF;
      clk_count := clk_count + 1;
      state <= send;
   ELSE
      clk_count := 0;
      if line = '1' then
          if ptr = 0 then
             line <= '0';
             state <= resetline;
          else
             ptr <= ptr - 1;
             state <= line1;
          end if;
      else
          if ptr = 0 then
             line <= '1';
             state <= resetline;
          else
             ptr <= ptr - 1;
             state <= line2;
          end if;
      end if;
   END IF;
```

```
END CASE;

--reset
IF(reset_n = '0') THEN
    state <= power_up;
END IF;

END IF;
END PROCESS;
END controller;
```

- **Code for TOP LEVEL File**

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
entity ControllerTest_TOP is
   port (
      clk          : in  std_logic;
      rst          : in  std_logic;
      lcd_e        : out std_logic;
      lcd_rs       : out std_logic;
      lcd_rw       : out std_logic;
      lcd_db       : out std_logic_vector(7 downto 0));

end ControllerTest_TOP;

architecture Behavioral of ControllerTest_TOP is

architecture Behavioral of ControllerTest_TOP is

   COMPONENT lcd_controller IS
     PORT(
       clk        : IN    STD_LOGIC;  --system clock
       reset_n    : IN    STD_LOGIC;  --active low reinitializes lcd
       rw, rs, e  : OUT   STD_LOGIC;  --read/write, setup/data, and enable for lcd
       lcd_data   : OUT   STD_LOGIC_VECTOR(7 DOWNTO 0); --data signals for lcd
       line1_buffer : IN STD_LOGIC_VECTOR(127 downto 0); -- Data for the top line of the LCD
       line2_buffer : IN STD_LOGIC_VECTOR(127 downto 0)); -- Data for the bottom line of the LCD
   END COMPONENT;

   -- These lines can be configured to be input from anything.
   -- 8 bits per character
   signal top_line : std_logic_vector(127 downto 0) := x"202043554920204C40484F5245202020"; --Translates to CUI LAHORE
   signal bottom_line : std_logic_vector(127 downto 0) := x"2020454345204445504052544D454E54"; -- ECE DEPARTMENT

begin

LCD: lcd_controller port map(
   clk => clk,
   reset_n => rst,
   e => lcd_e,
   rs => lcd_rs,
   rw => lcd_rw,
   lcd_data => lcd_db,
   line1_buffer => top_line,
   line2_buffer => bottom_line
);

end Behavioral;
```

Pre-Lab Task

You are provided with VHDL files implementing:

- lcd_controller.vhd (FSM-based LCD controller)
- top_lcd.vhd (Top-level wrapper)

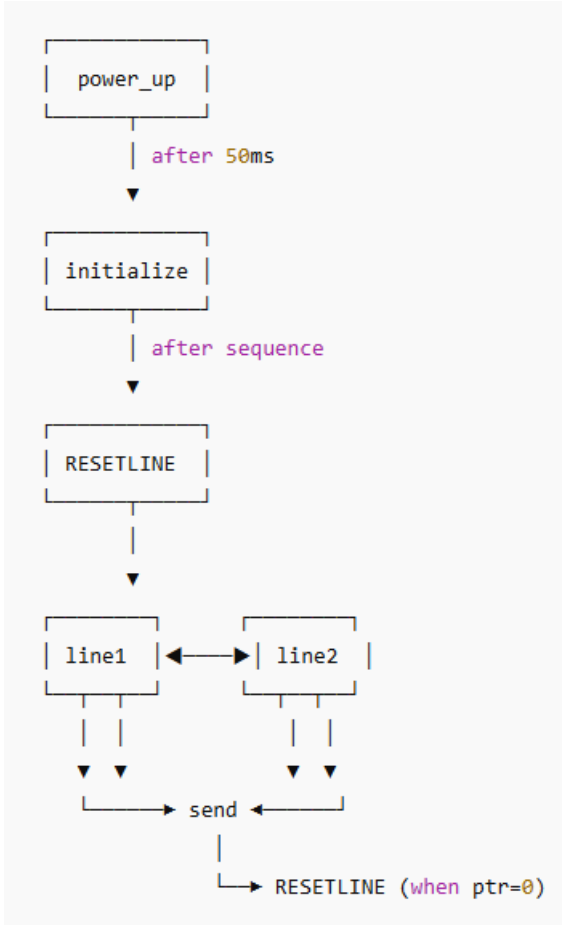
Create a new VHDL project in Quartus and test the given code on the FPGA board.

FSM Description

The LCD controller operates through six major states:

State	Description
power_up	Waits 50 ms to ensure power stabilization.
initialize	Executes LCD initialization sequence (Function Set, Display On/Off, Clear Display, Entry Mode Set).
RESETLINE	Sets the LCD cursor to the start of Line 1 or Line 2.
line1	Sends characters from Line 1 buffer to LCD.
line2	Sends characters from Line 2 buffer to LCD.
Send	Generates enable pulses to transmit data/commands to LCD.

The FSM to initialize LCD and display data is given below



Lab Tasks

Task 1: Push Button Controlled Display

Modify the top module to include a push button. When the button is pressed, the FSM should start data transmission to the LCD. Verify that the LCD correctly initializes and displays data when the button is pressed.

Task 2: Hex to ASCII Conversion

Write a VHDL function that converts a 32-bit hexadecimal input into 16 ASCII characters (128 bits total). Use this function to display hexadecimal values as readable ASCII on the LCD.

Task 3: Display Two Lines of Text

Modify the FSM-based controller to display two different strings:

- Line 1: Static or custom message (e.g., 'LCD FSM TEST')
- Line 2: Output from the Hex-to-ASCII conversion function
- Whenever the button is pressed, both lines should refresh according to the FSM sequence.

Rubric for lab assessment

The student performance for the assigned task during the lab session was:			
Excellent	The student completed all the tasks and showed the results without any help of the instructor.	4	
Good	The student completed all the tasks and showed the results with minimal help of the instructor.	3	
Average	The student partially completed the task and showed results.	2	
Worst	The student did not complete the task.	1	

Instructor Signature:_____Date: _____